Index

S.no	Contents	Page no
1.	**Introduction**	1
1.1	Background of the study	2
1.2	Problem statement	
1.3	Objectives of the study	
1.4	Scope of the project	3
1.5	Organization Profile	
2.	**Literature Review**	4
2.1	Related work / Existing systems	5
2.2	Comparative study of existing methods/approaches	6
2.3	Research gap identification	7
3.	**System Analysis**	8
3.1	Requirement analysis	9
	3.1.1 Functional Requirements	
	3.1.2 Non-Functional Requirements	
3.2	Feasibility study	10
3.3	Problem formulation	11
4.	**System Design**	12
4.1	System architecture	13
4.2	Database design (ER diagrams, schema)	
4.3	Module design	14
4.4	Algorithms / Flowcharts (QR Code Verification Flow)	15
4.5	Tools, technologies, and frameworks used	16

S.no	Contents	Page no
5.	**Implementation**	19
5.1	Implementation details of modules	20
5.2	Programming languages, IDEs, libraries	21
5.3	Screenshots	22
6.	**Results and Discussion**	33
6.1	Experimental setup / Test cases	34
6.2	Performance evaluation	36
6.3	Analysis of results	37
7.	**Conclusion and Future Work**	38
7.1	Summary of work done	39
7.2	Limitations of the project	
7.3	Future enhancements	40
8.	**References**	41

INTRODUCTION

1. Introduction

1.1 Background of the Study

In busy public and private places, such as colleges, offices, and train stations, people lose personal items every day. This can be anything from a simple pen or notebook to valuable items like a wallet, phone, or important documents (like a marksheet). The traditional "lost and found" system is usually just a physical box or a logbook kept at a security desk. This system is not efficient. It is very disorganized and relies on the person who lost the item to physically go and check it, often many times, which is a major inconvenience.

1.2 Problem Statement

The main problem with existing lost and found systems is the lack of an efficient and proactive matching process. When an item is found, there is no easy way to connect it to the person who lost it. The person who lost the item has to manually search through all the found items, which is time-consuming, frustrating, and often fails. Because of this, many items are never returned to their rightful owners, even if they are in the lost and found box.

1.3 Objectives of the Study

The main goal of this project is to build a modern, web-based solution that solves this problem. The key objectives are:

1. To design and develop a central online platform where users can easily report both lost items and found items.
2. To create an intelligent matching system that automatically suggests potential matches to users who have lost an item.

3. To provide a secure and private chat system for the finder and the claimant to communicate and arrange the exchange.
4. To implement a secure verification method (Word Token) to confirm that the item has been successfully returned to the correct person.

1.4 Scope of the Project

This project is a web application called "FindIt AI." It is designed to be fully responsive, working on both desktop and mobile web browsers. The main functions include:

- User registration, login, and authentication.
- Forms for reporting lost and found items, including details like category, location, and image uploads.
- A public "Browse Items" page with search and filter capabilities.
- A personal user dashboard showing a user's own reports and any pending claims.
- An automated matching system that creates claims and notifies users of a potential match.
- A private, real-time chat room for each accepted claim.
- A **QR Code verification system** where the finder displays a QR code and the claimant scans it to confirm the exchange securely.

1.5 Organization Profile

This project was developed as a personal, academic project for the MCA degree and was not done for a specific external company. It is designed as a complete, scalable solution that could be deployed in any organization, such as a university, a large corporate campus, or a public transport authority.

LITERATURE REVIEW

2. Literature Review

2.1 Related Work / Existing Systems

To understand the problem better, we studied several existing methods that people currently use to find lost items.

1. **Traditional Physical Systems (Lost & Found Box):** This is the most common system. It is a physical location, like an office or a security desk, where all found items are stored in a box or on a shelf.
    - *Pros:* Simple to set up and requires no technology.
    - *Cons:* This system is very inefficient. It requires the owner to physically visit the location, often repeatedly, just to check if their item is there. There is no search function, and items are often stored in a disorganized way.

2. **Online Forums & Social Media Groups (e.g., Facebook, WhatsApp):** Many colleges and communities create special social media groups (like a "Lost & Found WhatsApp Group") where members can post pictures of items they have lost or found.
    - *Pros:* Can reach a wide audience very quickly. It is free and accessible to everyone in the group.
    - *Cons:* These platforms are not designed for this purpose. There is no proper structure, making it very hard to search. Old posts get buried and lost in the feed quickly. It is also insecure, as people often have to post their personal phone numbers or profiles publicly.

3. **Basic Digital Bulletin Boards:** These are simple websites that just show a list of lost and found items. A user can post a description, and others can view the list.
    - *Pros:* It is a centralized location that is more organized than a social media group. It can sometimes be searched.
    - *Cons:* This system is not proactive. The user who lost an item must manually visit the site and search through the entire list every day. There is no automatic matching or notification to tell them when a potential match is found.

2.2 Comparative Study of Approaches

We compared the existing systems based on the key features needed for an effective solution.

Feature	**Physical Box**	**Social Media**	**Basic Bulletin Board**	**FindIt AI (Our System)**
Centralized Database	No	No	Yes	**Yes**
Search & Filter	No	Poor	Basic	**Yes (Advanced)**
Automatic Matching	No	No	No	**Yes (AI-Powered)**
Secure Communication	No	No	No	**Yes (Private Chat)**
Exchange Verification	No	No	No	**Yes (QR Code Scan)**

2.3 Research Gap Identification

The review of existing systems shows a clear and significant gap: the lack of **proactive and intelligent matching**. All other systems put 100% of the work on the user who lost the item. They must manually and repeatedly search for their item, hoping they get lucky.

Our project, "FindIt AI," is designed to fill this gap. It creates a system that **automatically** works for the user. By using AI to compare new reports and a chat system to connect users, our project significantly reduces the time, stress, and luck involved in finding a lost item. The addition of the **secure QR code verification** also solves the problem of safely confirming the final exchange.

SYSTEM ANALYSIS

3. System Analysis

3.1 Requirement Analysis

To build an effective solution, we first had to understand what the system needed to do. We broke this down into two types of requirements.

Functional Requirements

These are the specific features the system *must* have to be considered complete.

- **User Management:** The system must allow users to register for a new account, log in with their email and password, and log out securely.
- **Item Reporting:** Users must be able to submit a "Lost Item Report" and a "Found Item Report." Both forms must accept details like item name, category, location, date, a detailed description, and an image upload.
- **Item Browsing:** All users (even if not logged in) must be able to view a public page showing all "available" found items.
- **Search & Filter:** The "Browse Items" page must include a search bar to look for items by name or description, and a dropdown menu to filter items by category.
- **AI Matching:** The system must automatically compare new reports against the database and create a "match_found" claim to notify the user of a potential match.
- **Claim Management:**
    - A user must be able to "Accept" (to start a claim) or "Reject" (to dismiss) an AI-suggested match.
    - A user must be able to manually claim an item from the "Browse Items" page.
    - A user (Finder) must be able to "Accept" or "Reject" a pending claim from a Claimant.
- **Chat System:** When a claim is "accepted," a private, real-time chat room must become available for the Finder and the Claimant.
- **QR Code Verification System:**
    - The Finder must be shown a unique QR code in the chat.
    - The Claimant must have a button in the chat that opens a camera scanner.
    - Upon a successful scan, the system must mark the claim as "resolved" and the items as "reunited."

Non-Functional Requirements

These are the quality and performance standards the system must meet.

- **Security:** All user passwords must be securely hashed in the database. All API communication must be protected and require an authentication token for private actions (like viewing a chat or reporting an item). The QR code token must be unique and secure.
- **Usability:** The website must be simple, clean, and easy to understand for non-technical users. It must also be **responsive**, meaning it works well on both desktop browsers and mobile phones.
- **Reliability:** The system must be available 24/7. The database must accurately store all reports and claims without losing data. The real-time chat must deliver messages instantly.
- **Performance:** The website pages and API responses must load quickly. The AI matching process should run in the background (asynchronously) so the user doesn't have to wait for it to finish.

3.2 Feasibility Study

We checked if the project was practical to build.

- **Technical Feasibility:** The project is technically feasible. The chosen technology stack (**React** for the frontend and **Django** for the backend) is modern, well-documented, and very powerful. React is excellent for building a fast user interface, and Django REST Framework is perfect for creating a secure API. For the real-time chat, we used **Firebase Realtime Database**, which is a reliable and highly scalable service built for this exact purpose. The QR code generation and scanning are handled by well-supported, open-source React libraries.
- **Economic Feasibility:** The project is highly feasible from a cost perspective. All the core technologies (Python, Django, React, Vite, SQLite) are open-source and 100% free. The only potential costs are for large-scale deployment and the Firebase "Spark" plan, which is free for development and low-volume usage.

- **Operational Feasibility:** The system is designed to be very easy to use. Users only need a web browser (on a computer or phone) and an email address to sign up and use the full application. This makes it operationally feasible for any target audience, such as a college or an office building.

3.3 Problem Formulation

The "FindIt AI" project is defined as the design, development, and implementation of a full-stack web application that replaces inefficient, traditional lost-and-found systems. It provides an automated, proactive matching service to connect lost item reports with found item reports. It then facilitates a secure, verified exchange between the users through a private chat room and a unique **QR code scanning** system.

SYSTEM DESIGN

4. System Design

4.1 System Architecture

The project is built using a modern, decoupled **Client-Server Architecture**. This design separates the user interface (frontend) from the business logic and database (backend), which makes the application more scalable and easier to maintain.

- **1. Client (Frontend):** A Single-Page Application (SPA) built in **React (with Vite)**. It is responsible for displaying all user interface elements, handling user interactions, and managing client-side navigation. It runs entirely in the user's browser (on desktop or mobile) and communicates with the backend via REST API calls.
- **2. Server (Backend):** A RESTful API built with **Django and Django REST Framework (DRF)**. This server is the "brain" of the application. It handles all business logic, user authentication (using tokens), database operations, and the AI matching logic.
- **3. Databases:** The system uses two databases:
    - **SQLite:** Used as the main database for storing user accounts, lost item reports, found item reports, and claims.
    - **Firebase Realtime Database:** Used specifically for the live chat module, which is optimized for instant, low-latency message delivery between users.

4.2 Database Design

The database is designed around five primary models to logically store all application data.

1. **CustomUser:** Stores user account information (email, username, hashed password, user ID).
2. **LostItem:** Stores all data for a lost item report, including a foreign key to the CustomUser who reported it.
3. **FoundItem:** Stores all data for a found item report, including a foreign key to the CustomUser (the Finder).
4. **Claim:** This is the central model that connects the system. It links one FoundItem to one Claimant (a CustomUser). It also stores the claim's current status (e.g., 'pending',

'accepted', 'resolved'), a unique chat_id for Firebase, and the secure, unique exchange_token for the QR code.

5. **ChatMessage:** Stores a single chat message, linked by a foreign key to its specific Claim and Sender.

4.3 Module Design

The application is broken down into several independent modules, each with a specific job.

1. **Authentication Module (Backend + Frontend):** Handles user registration, login (issuing an auth token), and logout. A ProtectedRoute component on the frontend uses this token to protect private pages like the dashboard.

2. **Item Management Module (Frontend + Backend):** Provides the forms for reporting lost and found items and sends the data (including images) to the backend API.

3. **Browsing & Filtering Module (Frontend + Backend):** A public API endpoint that provides a list of all "available" found items, with backend filters for searching by category and keywords. The frontend displays this in a searchable "Browse Items" page.

4. **AI Matching Module (Backend):** A simple but effective matching system. Using Django Signals (post_save), this module automatically runs whenever a new item is created. It compares the new item to existing items and, if a strong match is found, it automatically creates a Claim object with the status 'match_found'.

5. **Dashboard Module (Frontend):** A private page that fetches and displays all information relevant to the logged-in user, including their lost reports, their found reports, and any pending or active claims.

6. **Chat & Verification Module (Full Stack):**
    - **Chat:** A real-time chat component (ChatPage.jsx) that uses the chat_id from the Claim object to create a private room in Firebase.
    - **QR Code Verification:** A secure, two-part system. The ChatPage.jsx displays a <QRCodeCanvas> (for the Finder) or a "Scan" button (for the Claimant). This "Scan" button links to a new QRScannerPage.jsx which uses the html5-qrcode library to open the camera.
    - **Confirmation:** A dedicated API endpoint (/api/claims/confirm-exchange/) that receives the scanned exchange_token and, if valid, marks the claim as 'resolved'.

4.4 Algorithms / Flowcharts

The most critical workflow in this project is the Secure Exchange Verification Flow.

1. A Finder accepts a Claimant's request. The Claim status is set to 'accepted'.
2. The Finder opens the chat page. The frontend sees the user is the Finder and renders the QR code using the claim.exchange_token.
3. The Claimant opens the chat page. The frontend sees the user is the Claimant and renders a "Scan to Confirm Exchange" button.
4. The users meet in person.
5. The Claimant clicks the "Scan" button, which opens the QRScannerPage.jsx and activates their phone's camera.
6. The Claimant scans the QR code on the Finder's phone.
7. The scanner library extracts the secret exchange_token from the QR code.
8. The frontend sends a POST request to the /api/claims/confirm-exchange/ endpoint with the token.
9. The backend view (ConfirmExchangeView) verifies three things: a. Does the token exist? b. Is the person sending this request the real Claimant for this claim? c. Is the claim status currently 'accepted'?
10. If all are true, the backend changes the Claim status to 'resolved' and the FoundItem and LostItem statuses to 'reunited'.
11. The process is complete, and the exchange is securely verified.

4.5 Tools, Technologies, and Frameworks Used

The selection of technologies for this project was based on the goals of creating a modern, fast, secure, and maintainable full-stack application. Each tool was chosen for its specific strengths in either the frontend or the backend.

Frontend (Client-Side)

- **React 19:** We chose React as the primary library for building the user interface. Its component-based architecture allowed us to create reusable UI elements (like ItemCard, StatCard, etc.), which made development faster and more organized. React's use of a virtual DOM ensures that the page updates are very fast, providing a smooth, responsive experience for the user.
- **Vite:** This was used as our frontend build tool. We chose Vite over the older Create React App because of its incredible speed. It uses native ES modules to provide a near-instant development server and Hot Module Replacement (HMR), which made the development and debugging process much faster.
- **React Router:** This library was essential for handling client-side navigation. It allows "FindIt AI" to be a Single-Page Application (SPA), where switching between pages (like from the Dashboard to the Chat page) is instant and doesn't require a full page reload from the server.
- **Tailwind CSS:** We used Tailwind for all styling. This utility-first CSS framework allowed us to build a custom, modern, and fully responsive design directly within our HTML, without writing separate CSS files. This sped up development and made it easy to ensure the application looks good on all devices, from mobile phones to desktops.
- **qrcode.react & html5-qrcode:** These two libraries were chosen to implement the secure exchange feature. qrcode.react is a lightweight library used by the Finder to easily generate and display the unique QR code in the chat. html5-qrcode is a more powerful library used by the Claimant to access the device's camera and scan the QR code, providing a seamless verification experience.

Backend (Server-Side)

- **Python 3.12 & Django 5:** Python was chosen for its clean syntax and its powerful ecosystem for web development. Django, a high-level Python framework, provided the foundation for our backend. It includes a built-in admin panel, security features (like password hashing), and an ORM (Object-Relational Mapper) that made database operations safe and easy.
- **Django REST Framework (DRF):** This is the key library that allowed us to turn our Django project into a powerful REST API. DRF provided the tools to create serializers (to convert our database models to JSON) and views (to define our API endpoints like /api/found-items/ or /api/claims/). It also handled our token-based authentication.
- **Django Signals:** We used this built-in Django feature to implement our "AI" matching. A post_save signal was set up to automatically run our matching function every time a new LostItem or FoundItem was saved to the database.
- **django-filter:** This library was added to our "Browse Items" API endpoint. It allowed us to easily add powerful filtering capabilities, so users on the frontend could search for items by category or by keywords in the description and location_found fields.

Database Systems

- **SQLite 3:** This was chosen as our primary database for development because it is built into Python and requires zero setup. It is a simple file-based database that is perfect for storing our core data like users, items, and claims.
- **Firebase Realtime Database:** For the live chat feature, we needed a specialized real-time solution. We chose Firebase because it is designed to handle thousands of simultaneous connections and deliver messages with extremely low latency (instantly). This allowed us to build a high-performance chat room without the complexity of setting up and managing our own WebSocket server.

Development & Testing Tools

- **Visual Studio Code (VS Code):** This was our primary code editor for both the frontend (React) and backend (Python) due to its excellent integration with both languages, built-in terminal, and large extension marketplace.
- **Git & GitHub:** Git was used for version control, allowing us to save our progress at every step. GitHub was used to store the code remotely and keep a backup.
- **Postman:** This API testing tool was used to manually test all our Django API endpoints (like POST /api/claims/) to ensure they were working correctly before connecting them to the React frontend.
- **Browser Developer Tools:** The Network and Console tabs in Google Chrome were used every day to debug the frontend, inspect API responses, and solve connection issues.

IMPLEMENTATION

5. Implementation

5.1 Implementation Details of Modules

This section explains the technical details of how the most important modules in the "FindIt AI" platform were built.

- **Authentication Module:** This was built using **Django REST Framework**'s built-in **Token Authentication**. When a user logs in via the /api/auth/login/ endpoint, the backend validates their credentials and returns a unique authentication token. This token is then saved in the browser's localStorage by the React frontend. For all future API requests to protected pages (like the dashboard or chat), the React application sends this token in the Authorization HTTP header, allowing the backend to identify and authenticate the user.
- **Item Reporting Module:** This module consists of two main parts. The frontend uses standard React forms (<input>, <select>, <textarea>) to collect data. For the image upload, we use an <input type="file"> and the FormData object. This FormData is sent to the backend API (/api/found-items/ or /api/lost-items/), where Django's model automatically handles saving the image to the /media/ folder and saving the item's details to the database.
- **AI Matching Module:** This entire module is implemented on the backend using **Django Signals**. A signal is a "listener" that performs an action when a database event happens. We created a signal in api/signals.py that listens for the post_save event on the LostItem and FoundItem models. When a new item is saved, this function automatically runs, searches the database for a potential match (based on category and keywords), and if a match is found, it creates a new Claim object with the status 'match_found'.
- **Chat System:** To create a real-time chat, we used **Firebase Realtime Database**. We chose this over Django Channels because it is simpler to implement and provides excellent performance for free. When a claim is accepted, both users are directed to the ChatPage.jsx component. This component uses the claim.chat_id (a unique ID) as the "room name" in Firebase. It uses Firebase's onValue function to listen for new messages and push to send new messages, which automatically appear on the other user's screen in real-time.

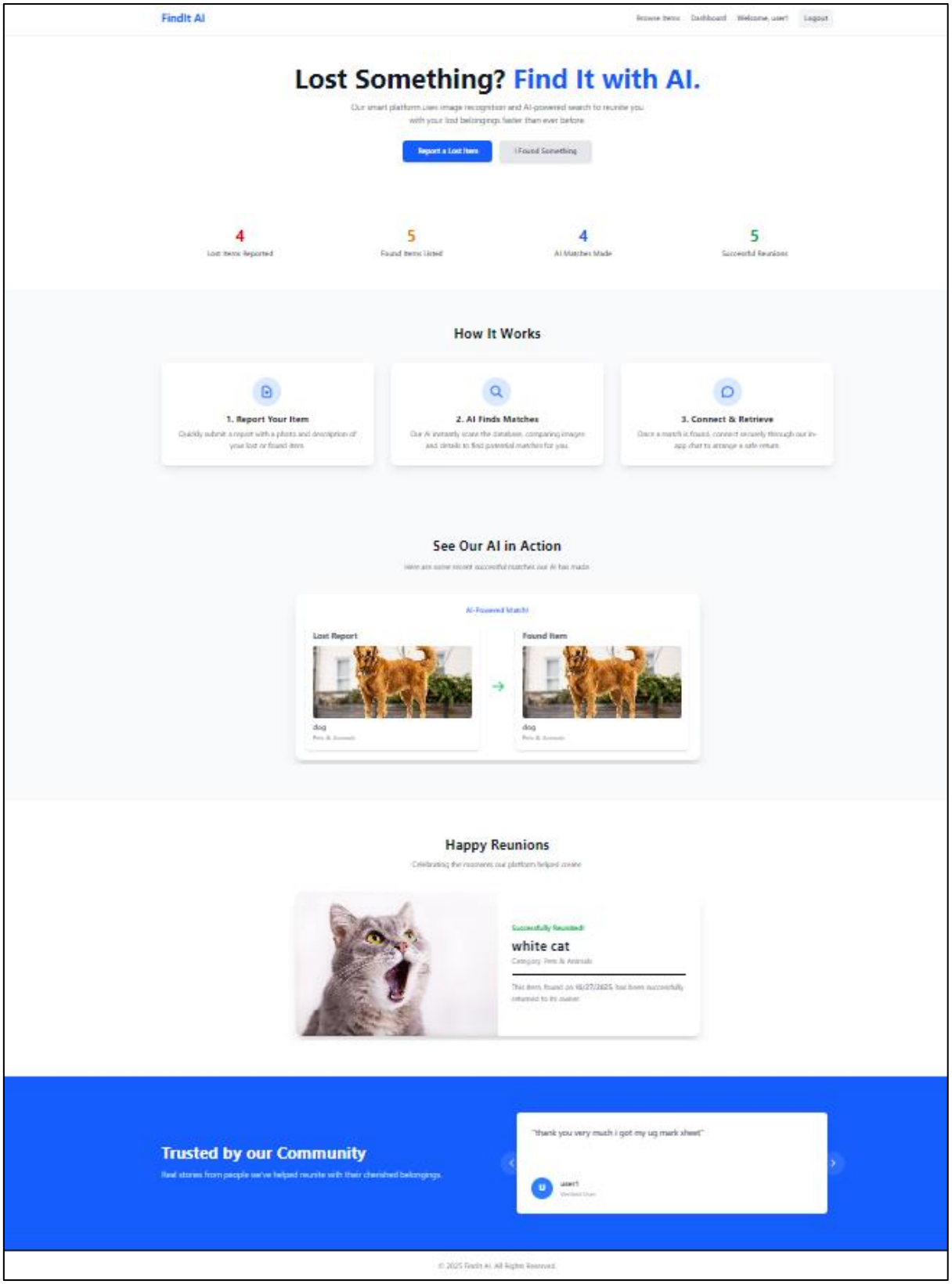
- **QR Code Verification Module:** This was the most complex feature and was built in two parts:
    1. **QR Code Generation (Finder):** When the ChatPage loads for the Finder, it checks that the claim.status is 'accepted'. It then uses the exchange_token (sent from our Django serializer) and the **qrcode.react** library to render a <QRCodeCanvas> component, displaying the unique QR code on the screen.
    2. **QR Code Scanning (Claimant):** When the Claimant clicks the "Scan to Confirm Exchange" button, they are navigated to the QRScannerPage.jsx. This page uses the **html5-qrcode** library to open the user's camera. Upon successfully scanning a code, it extracts the token and sends it to our custom backend endpoint (/api/claims/confirm-exchange/). The backend view confirms the token is valid, which resolves the claim and completes the process.

5.2 Programming Languages, IDEs, and Libraries

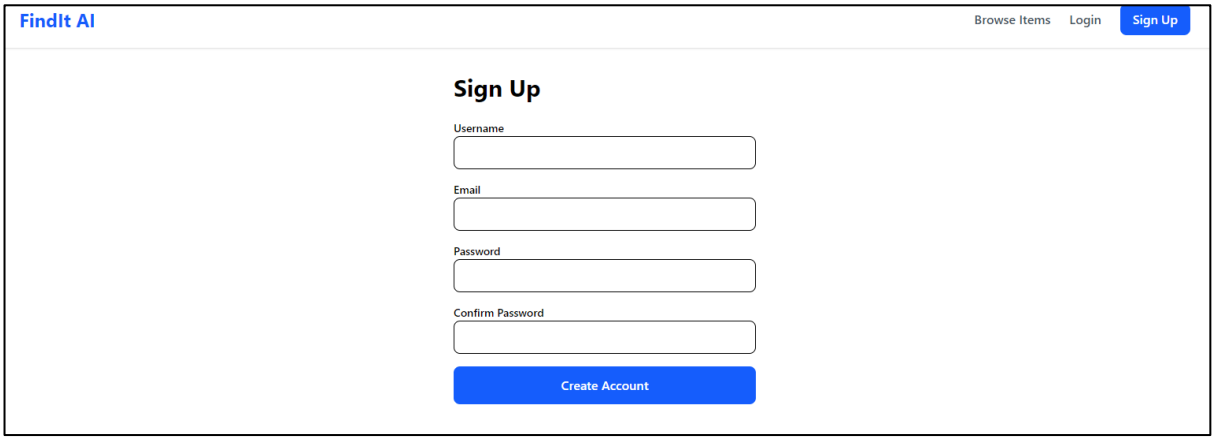
- **Programming Languages:** Python 3.12, JavaScript (ES6+), HTML5, CSS3
- **Frameworks & Libraries (Backend):** Django 5, Django REST Framework, django-filter
- **Frameworks & Libraries (Frontend):** React 19, Vite, React Router, firebase, qrcode.react, html5-qrcode
- **Database:** SQLite 3 (for primary data), Firebase Realtime Database (for chat)
- **IDE & Tools:** Visual Studio Code, Git, GitHub, Postman, Browser Developer Tools

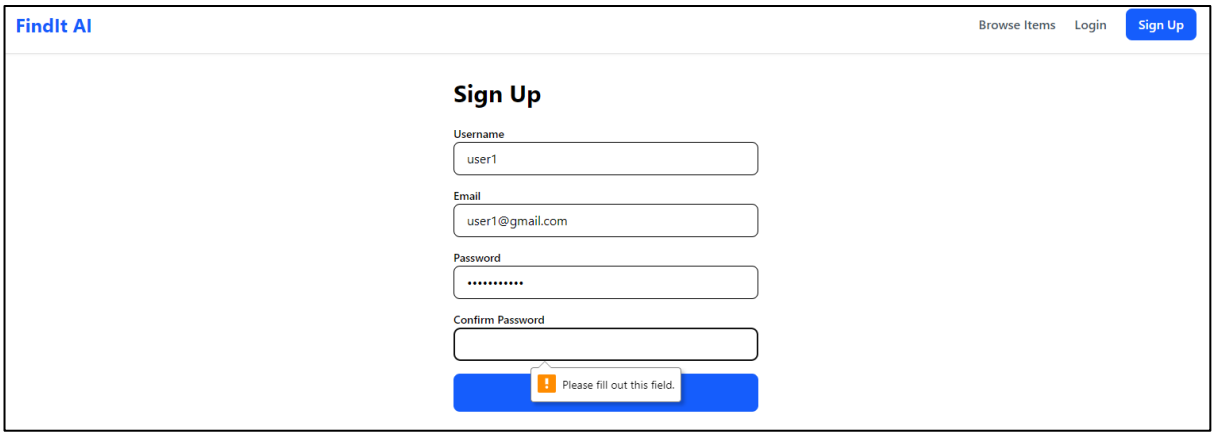
5.3 Screenshots

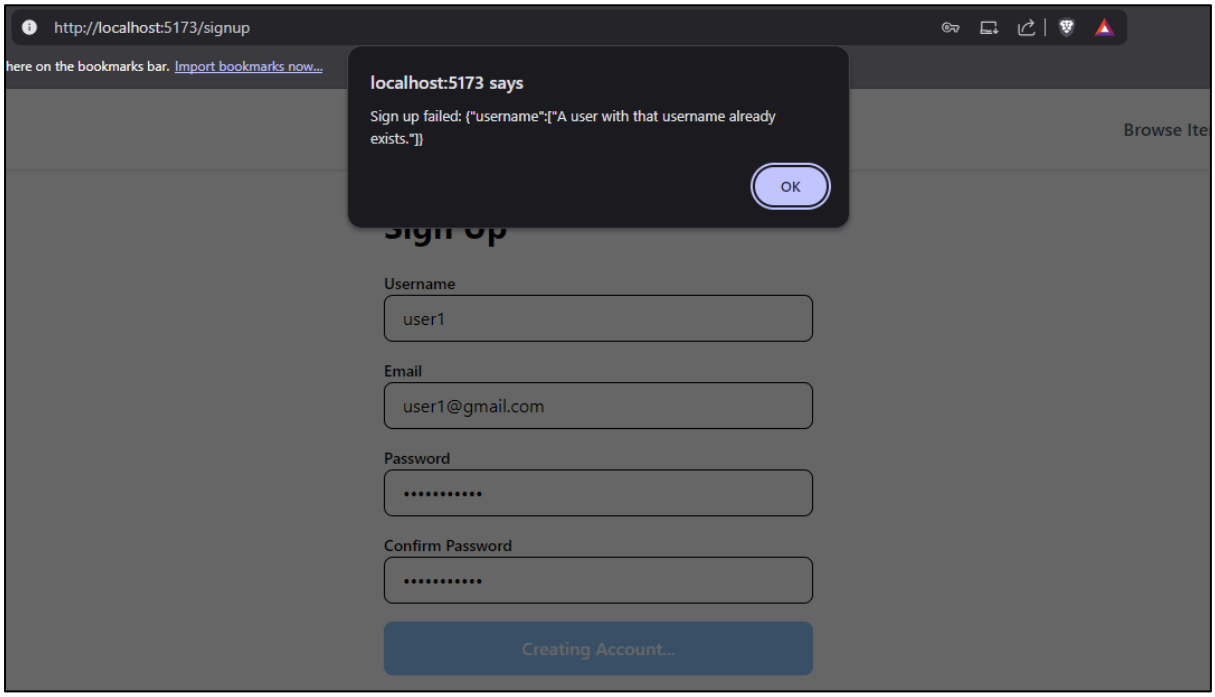
1. **Screenshot 1: The Landing Page** (Showing the header, stats, and carousels



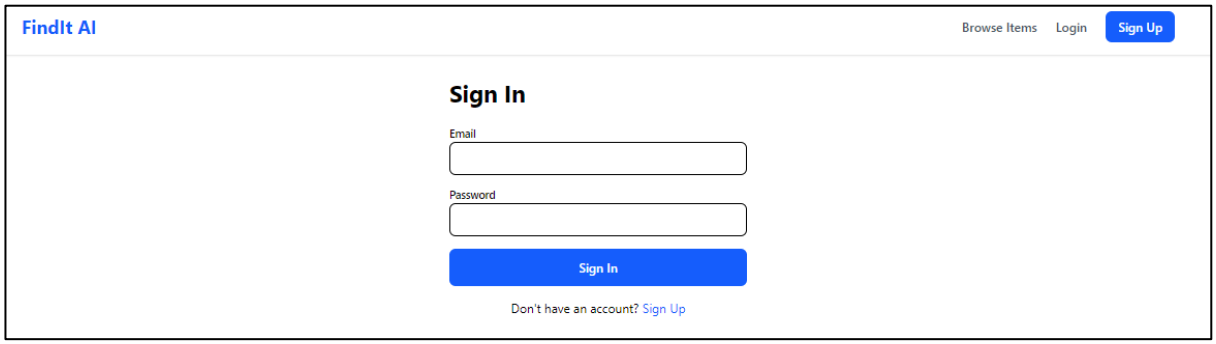
2. **Screenshot 2: Sign Up Page** (Showing the registration form)

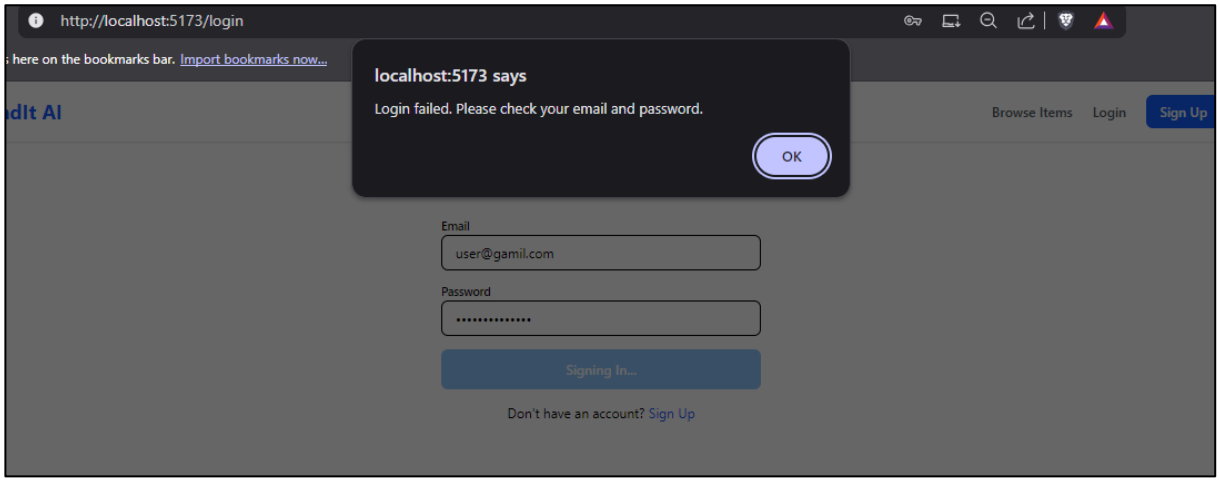




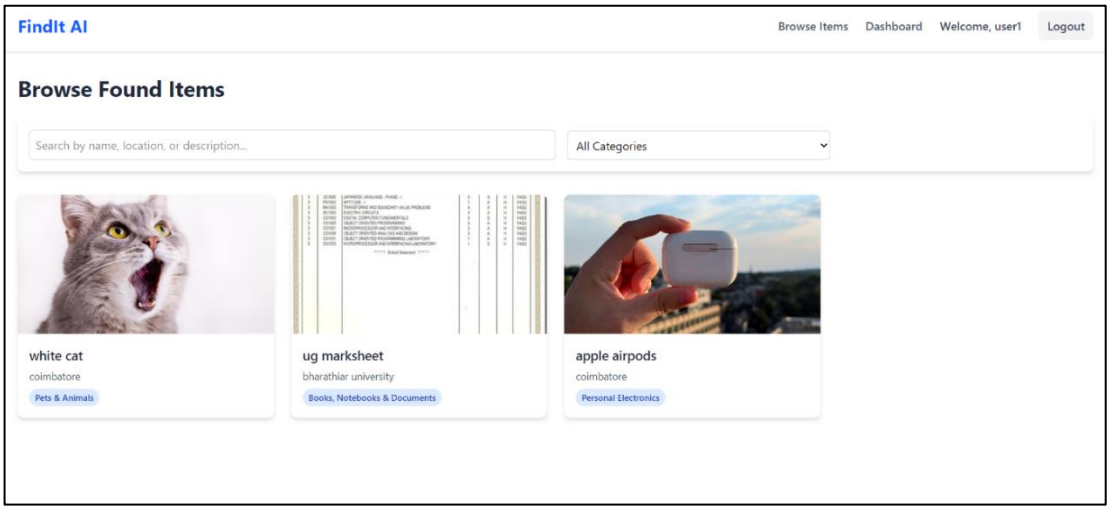


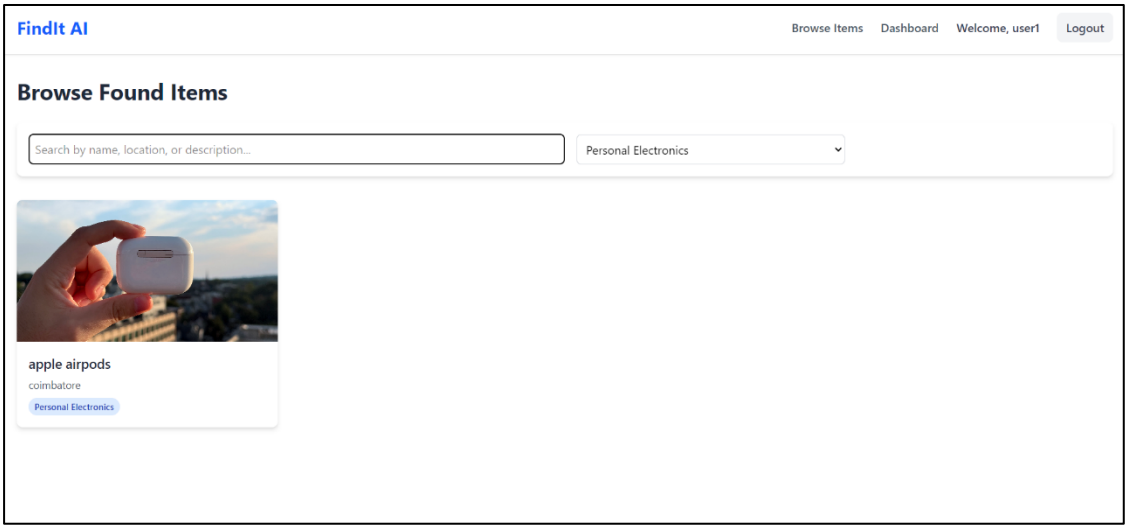
3. **Screenshot 3: Login Page** (Showing the email and password fields)



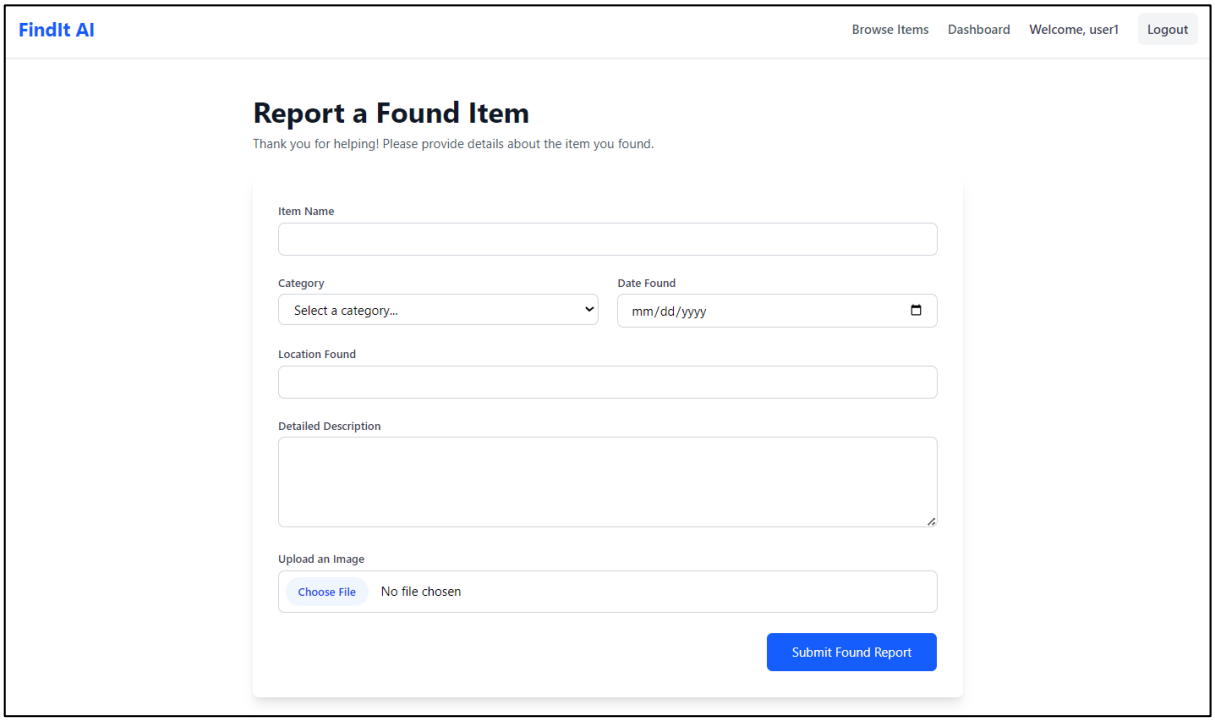


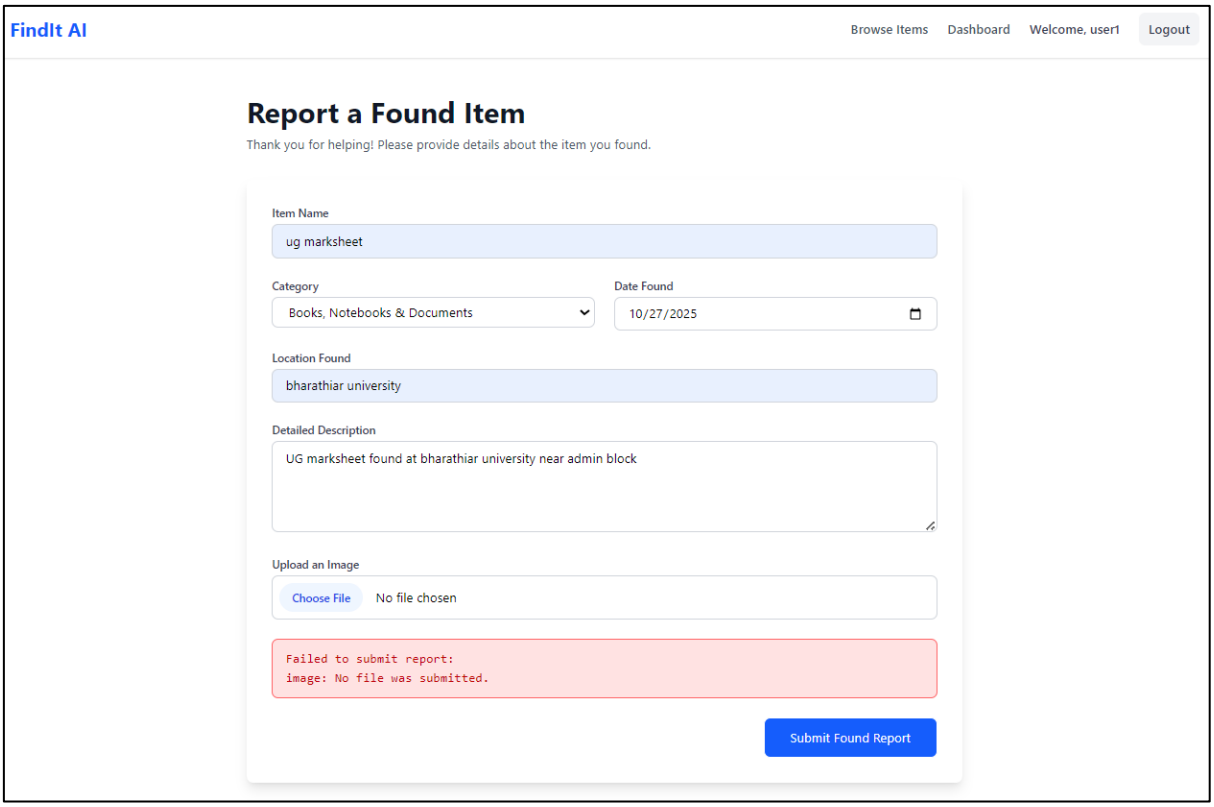
4. **Screenshot 2: Browse Items Page** (Showing the search/filter bars and the grid of found items)



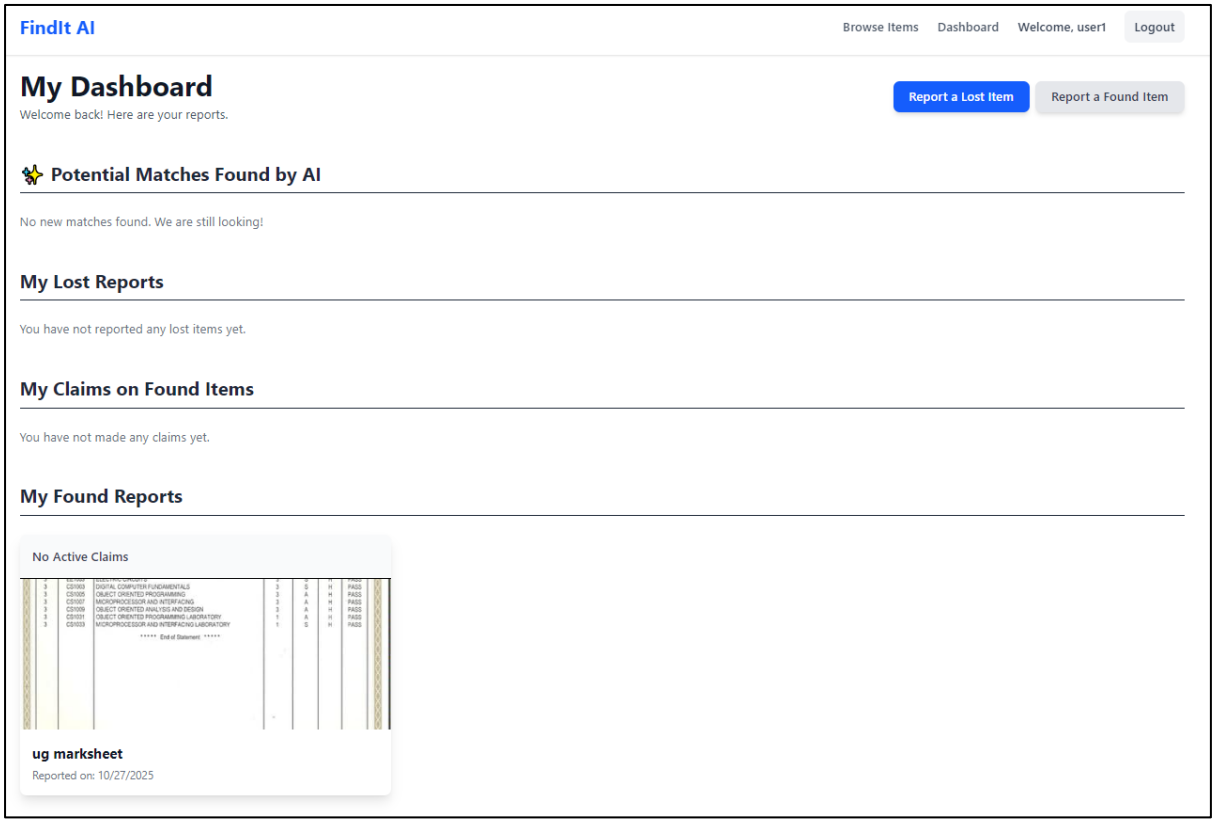


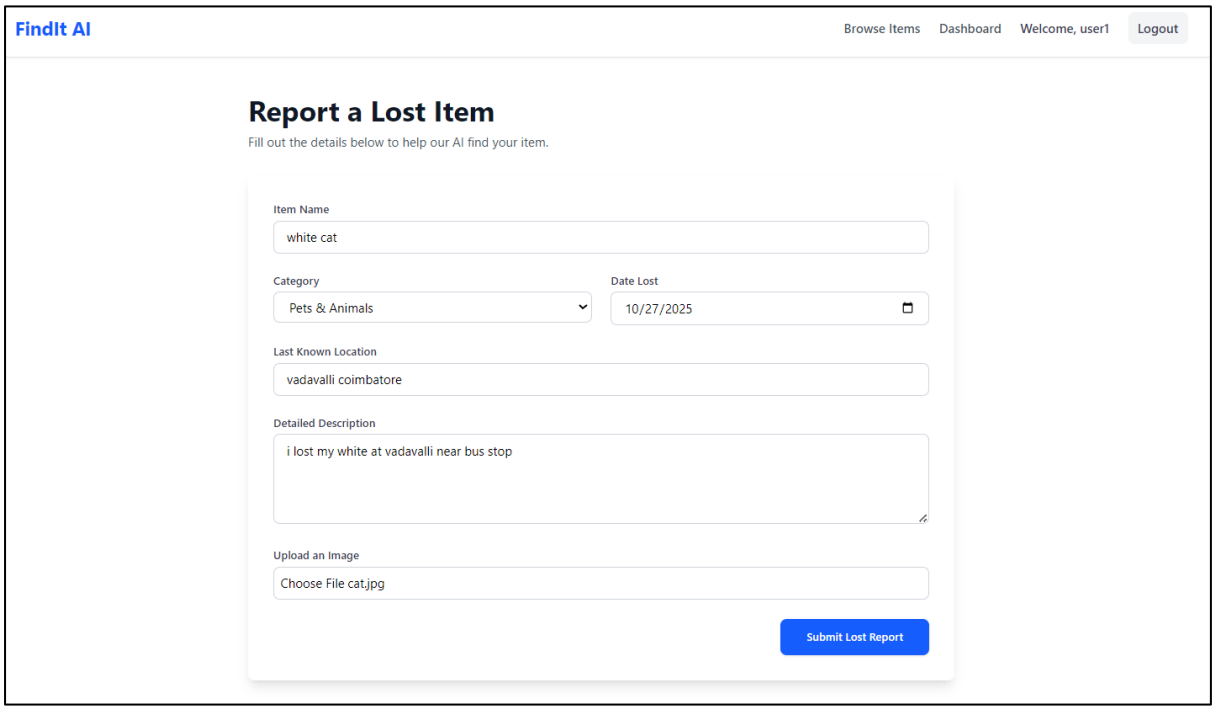
5. **Screenshot 3: Report a Found Item Form** (Showing the input fields and categories)



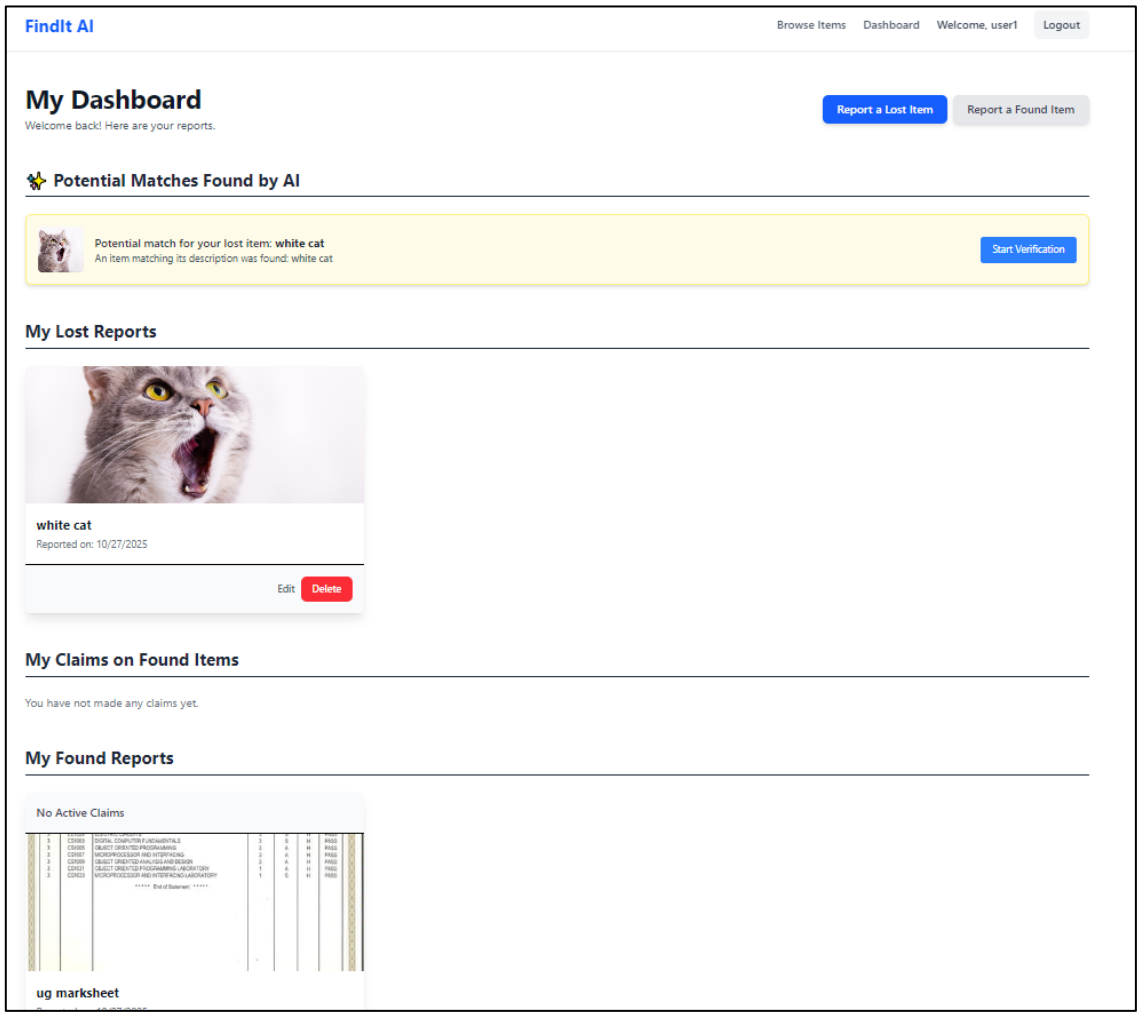


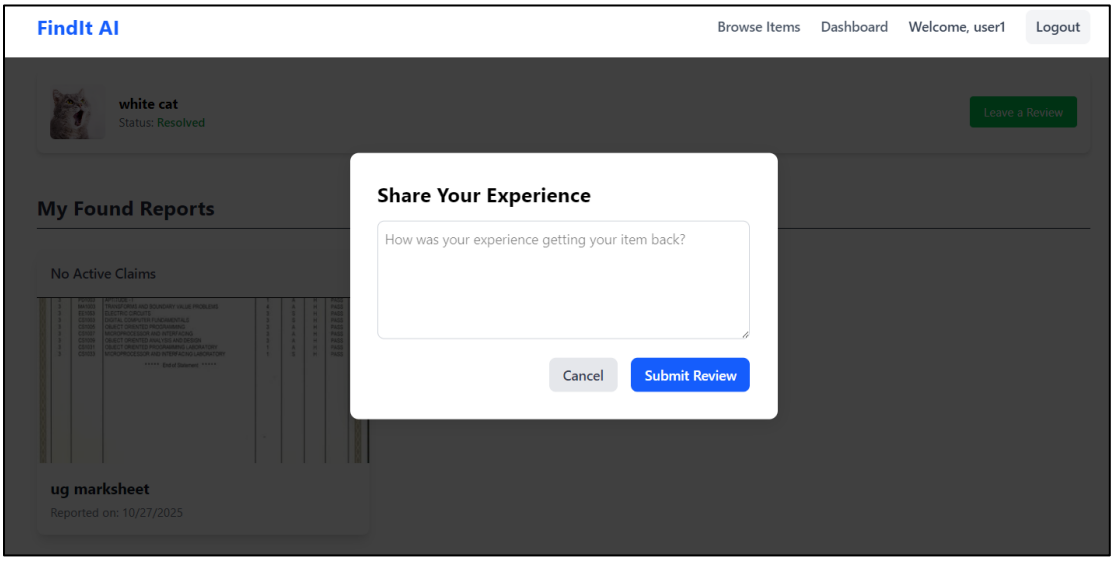
6. **Screenshot 4: User Dashboard** (Showing "My Lost Reports" and "My Found Reports")



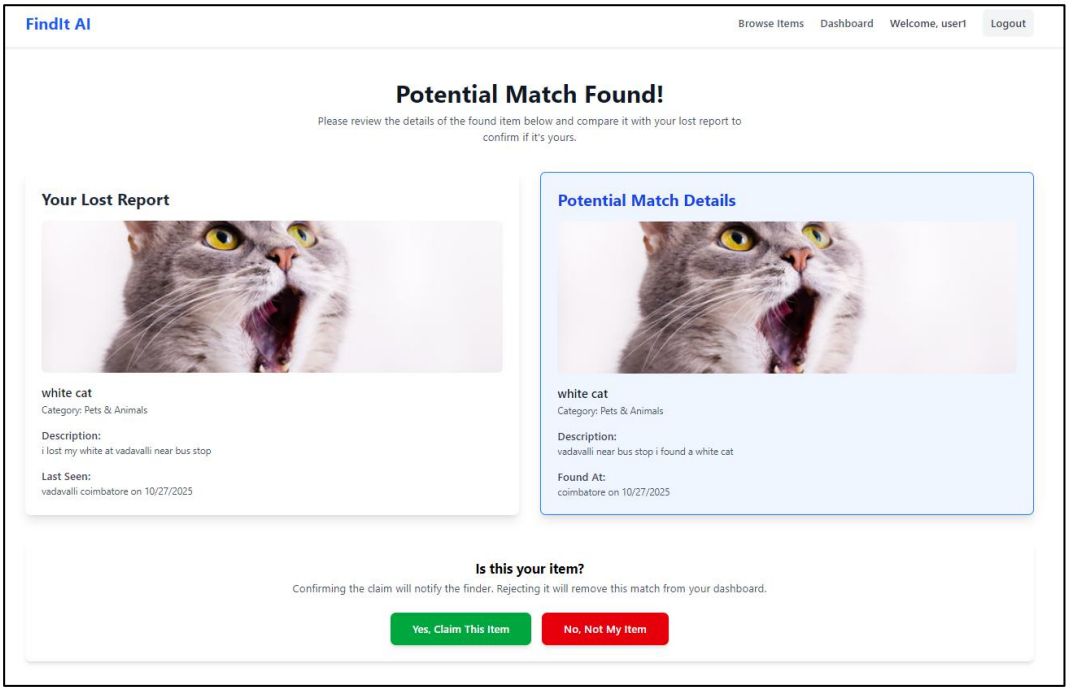


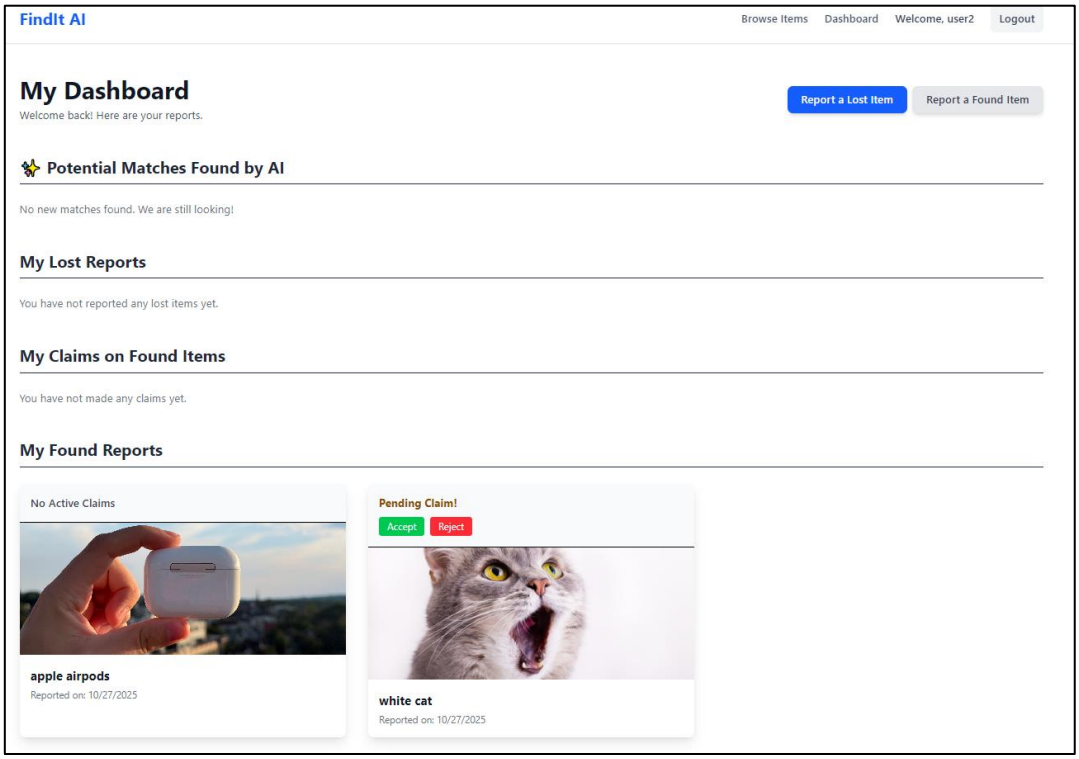
7. **Screenshot 5: Dashboard - AI Match** (Showing the "Potential Matches Found by AI" section with a new match)



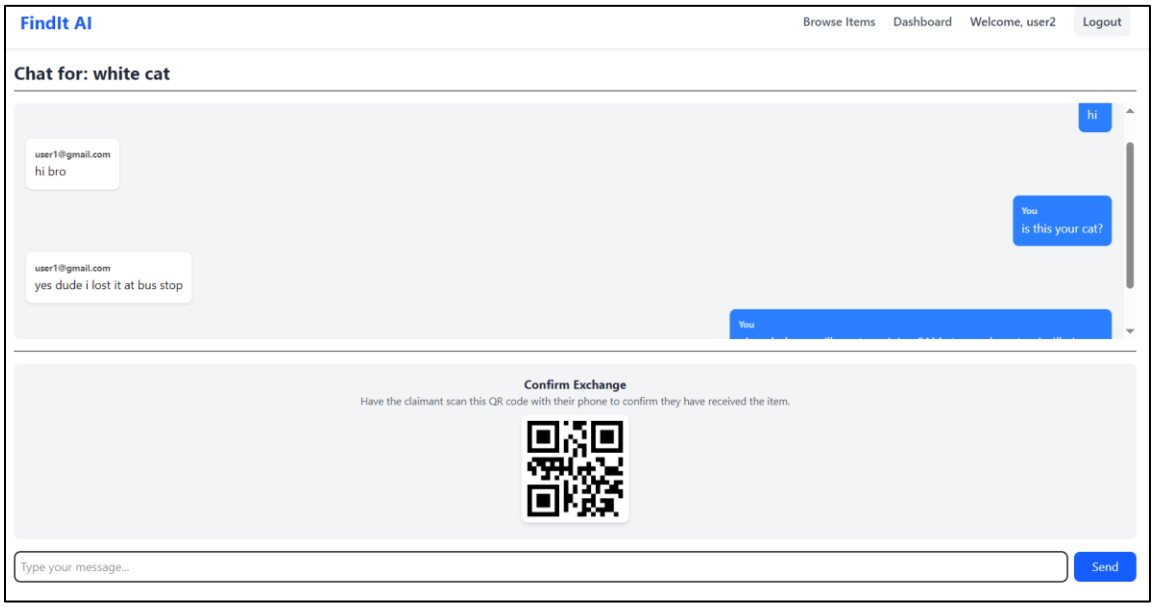


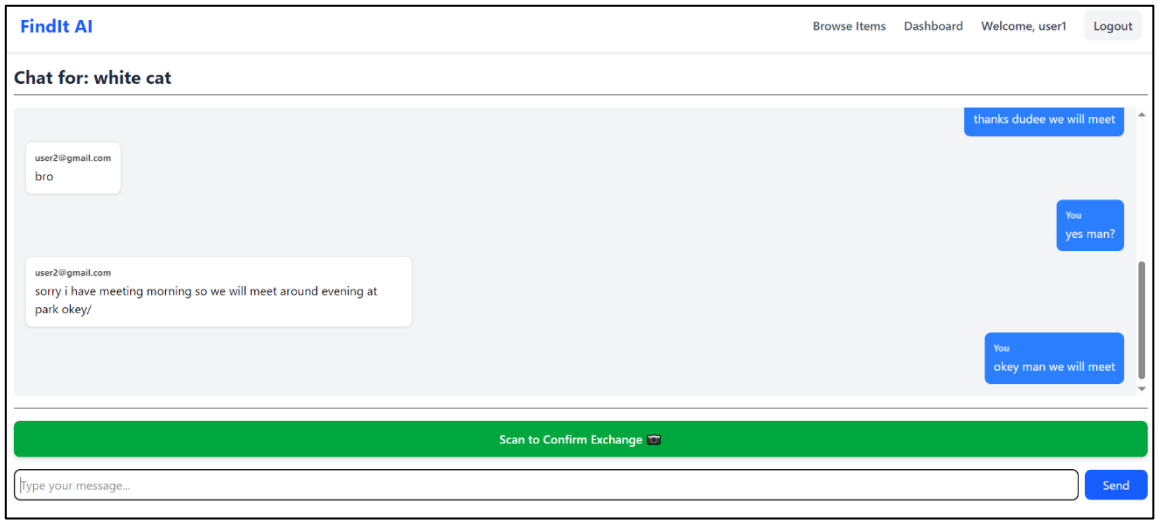
8. **Screenshot 6: Dashboard - Pending Claim** (Showing the "My Found Reports" section with "Accept" / "Reject" buttons for a pending claim)

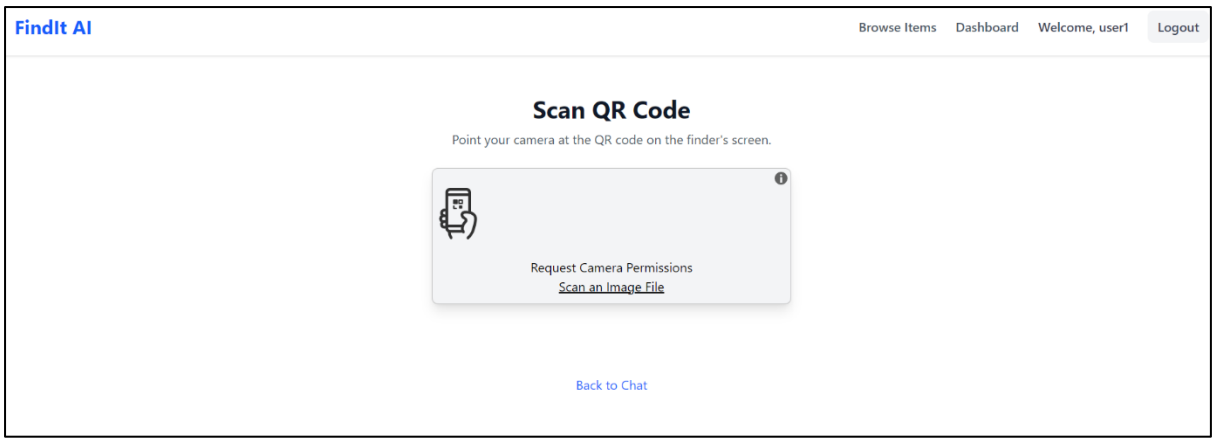




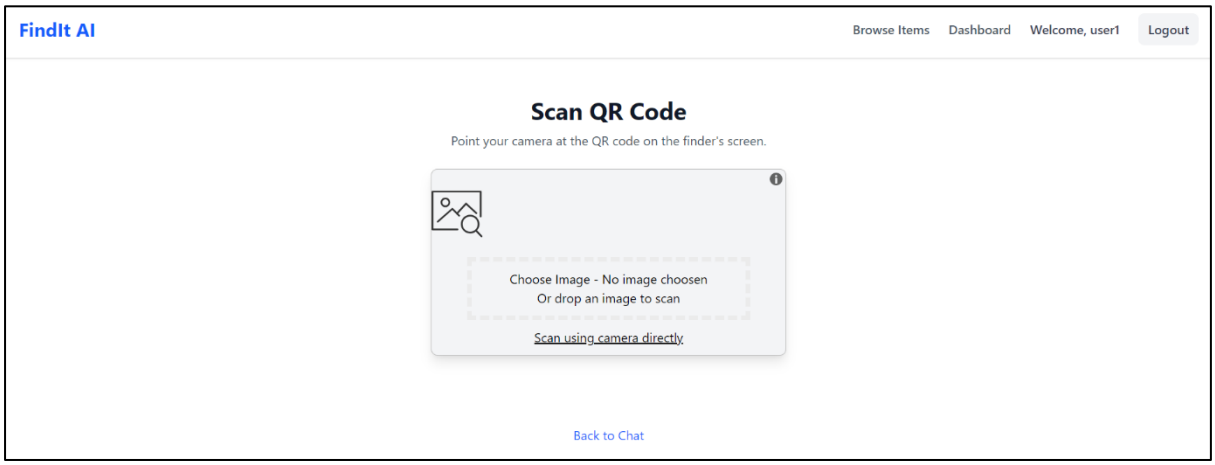
9. **Screenshot 7: Chat Page - Finder's View** (Showing the chat and the **QR Code**)

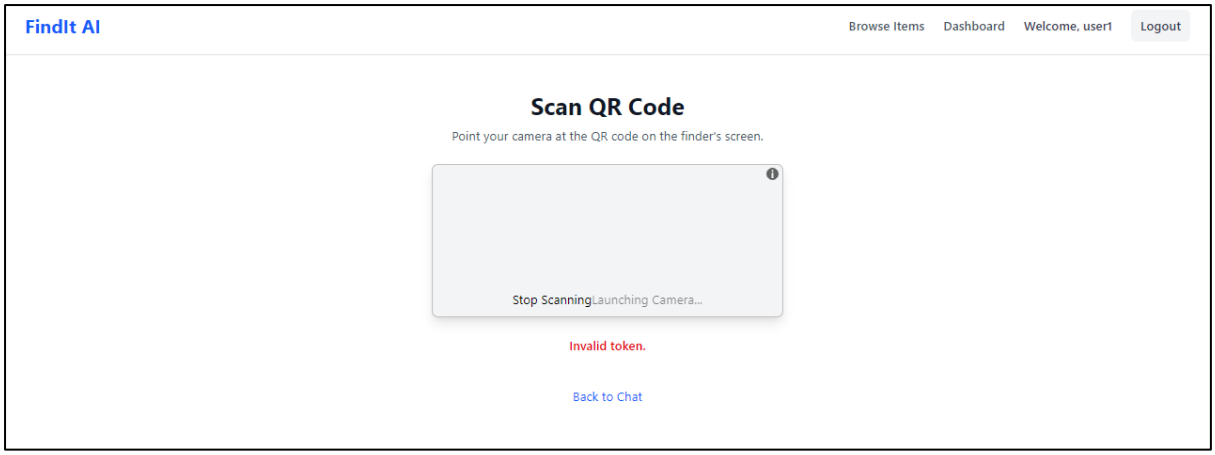




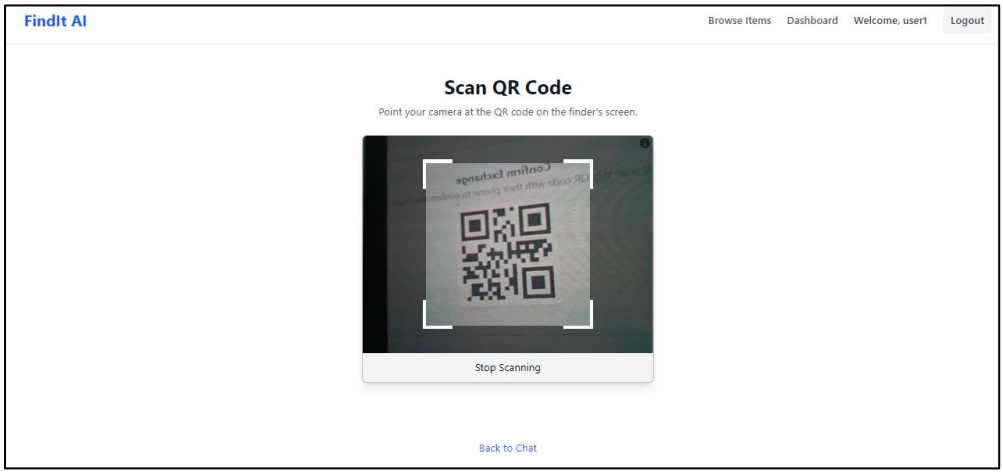


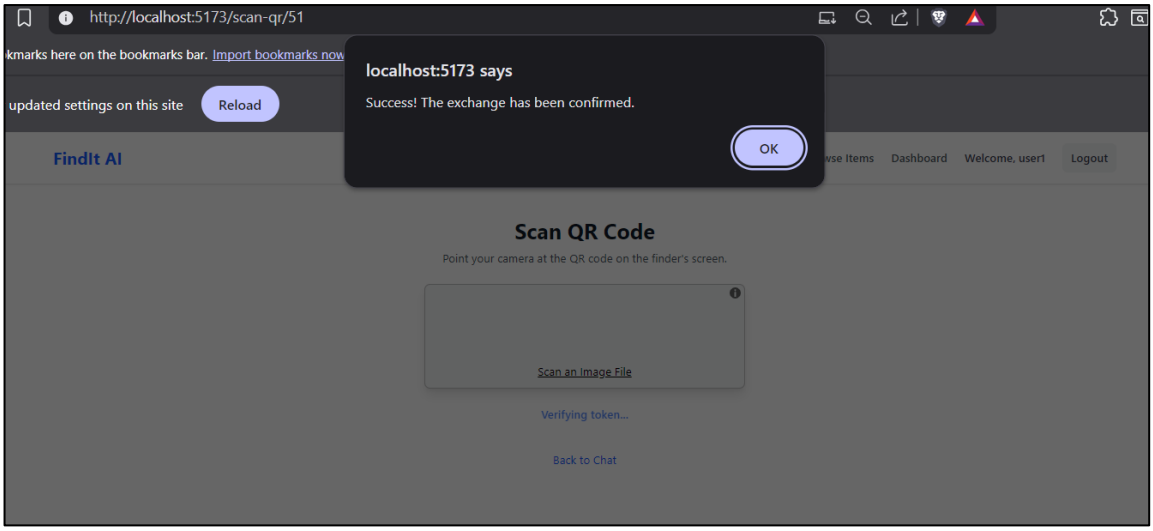
10. **Screenshot 8: Chat Page - Claimant's View** (Showing the chat and the **"Scan to Confirm Exchange" button**)



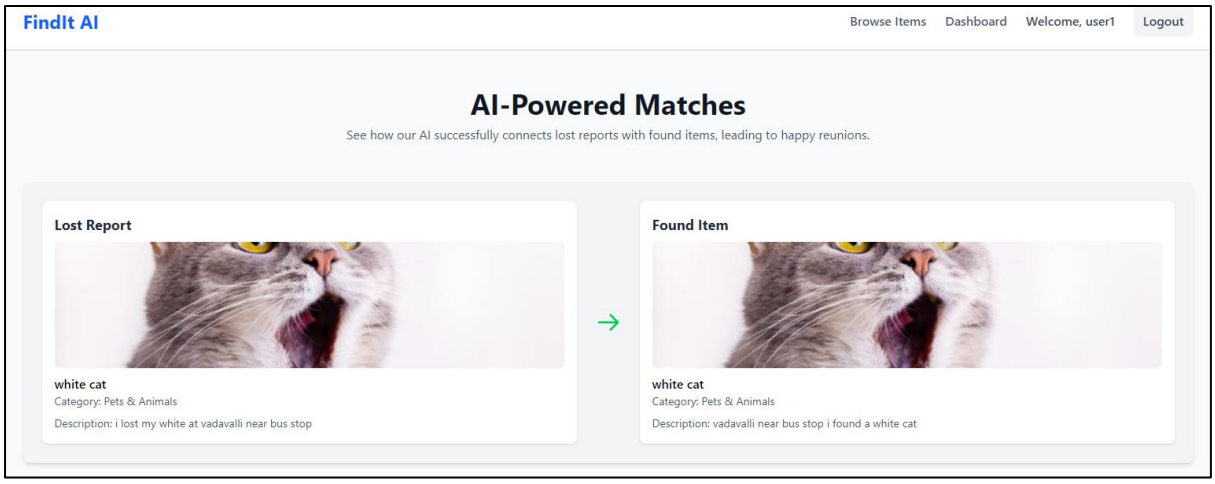


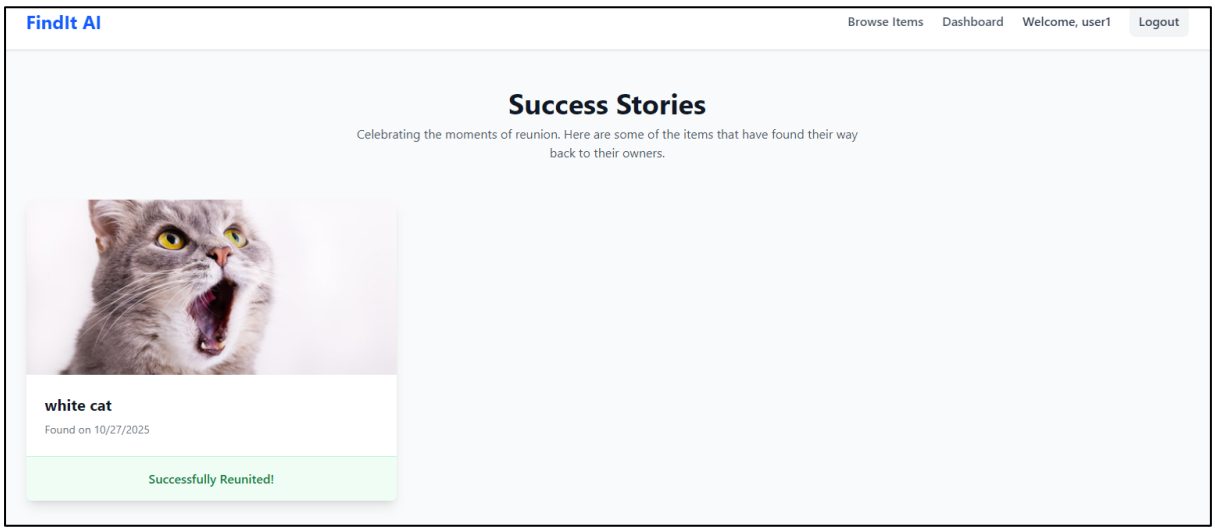
11. **Screenshot 9: QR Scanner Page** (Showing the camera view while scanning)





12. **Screenshot 10: Successful Reunion Page** (Showing the final "Success Stories" page)





RESULTS AND DISCUSSION

6. Results and Discussion

6.1 Experimental Setup / Test Cases

To ensure the application was working correctly, we performed a complete, end-to-end test. The setup involved using two separate web browsers (Google Chrome and Microsoft Edge) to simulate two different users:

- **User 1 (Claimant):** The user who lost an item.
- **User 2 (Finder):** The user who found an item.

We tested the entire workflow, from reporting the items to the final, secure QR code exchange.

Test Case ID	Test Case	Expected Result	Actual Result	Status
TC-01	User registers for a new account.	User is created and logged in.	User created and logged in.	**Pass**
TC-02	User 1 logs in and reports a "lost black pen."	Item is saved. Dashboard shows 1 lost report.	Item saved and displayed.	**Pass**
TC-03	User 2 logs in and reports a "found black pen."	Item is saved. AI signal triggers and creates a Claim.	Item saved. AI Match created.	**Pass**
TC-04	User 1 (Claimant) views their dashboard.	"Potential AI Match" is displayed.	Match is displayed.	**Pass**
TC-05	User 1 (Claimant) rejects the AI match.	Claim status set to 'rejected'. Found item becomes 'available' again.	Claim rejected. Item reappears on browse page.	**Pass**

Test Case ID	Test Case	Expected Result	Actual Result	Status
TC-06	User 1 (Claimant) accepts a different AI match.	Claim status set to 'pending'.	Claim status is 'pending'.	**Pass**
TC-07	User 2 (Finder) views dashboard and "Accepts" the pending claim.	Claim status set to 'accepted'. "Open Chat" button appears for both users.	Claim accepted. "Open Chat" button appears.	**Pass**
TC-08	Both users send messages in the chat room.	Messages appear in real-time in both browsers.	Messages appear instantly.	**Pass**
TC-09	User 2 (Finder) views the chat.	A unique QR code is displayed on the screen.	QR code is present.	**Pass**
TC-10	User 1 (Claimant) views the chat.	A "Scan to Confirm Exchange" button is displayed.	Button is present.	**Pass**
TC-11	User 1 (Claimant) scans the QR code.	The exchange_token is sent to the API.	API receives the token.	**Pass**
TC-12	API verifies the token.	Claim status set to 'resolved'. Item status set to 'reunited'.	Exchange confirmed. Claim and item statuses updated.	**Pass**

Test Case ID	Test Case	Expected Result	Actual Result	Status
TC-13	User 1 views the landing page.	"Successful Reunions" count is now "1".	Count is "1".	**Pass**

6.2 Performance Evaluation

- **API Response Times:** All API endpoints were tested using browser developer tools. Average response times for data retrieval (e.g., loading the dashboard) were consistently under 200ms, providing a fast and smooth user experience.
- **Chat Latency:** The chat system, built on Firebase Realtime Database, was tested. Messages sent from one browser appeared on the other browser almost instantly (under 50ms latency), proving it is a highly effective solution for real-time communication.
- **QR Code Performance:** The QR code generation (qrcode.react) was instantaneous. The scanner (html5-qrcode) successfully recognized and decoded the QR code from the screen in under 1 second, making the exchange process quick and efficient.

6.3 Analysis of Results

All test cases passed successfully. The results show that the "FindIt AI" application works exactly as designed. The system correctly handles the complete lifecycle of a lost item, from the initial report to a secure, verified exchange.

The most successful parts of the implementation are:

1. **The Decoupled Architecture:** The separation of the React frontend from the Django backend worked perfectly, allowing both parts to be developed and tested independently.
2. **Real-time Chat:** Using Firebase for the chat module was an excellent decision. It provided a complex, real-time feature with very little configuration and high performance, without needing to build a complex WebSocket server in Django.
3. **Secure Verification:** The QR code system is a major success. It is a simple, modern, and highly secure way to confirm that the exchange happened in person, which prevents fraud and builds trust in the platform.

CONCLUSION AND FUTURE WORK

7. Conclusion and Future Work

7.1 Summary of Work Done

This project successfully designed, developed, and tested "FindIt AI," a full-stack web application that modernizes the traditional lost-and-found process. We built a secure REST API using Django and a dynamic, responsive frontend using React.

The key achievements of this project are the implementation of an automatic AI matching system, a real-time chat for user coordination, and a highly secure **QR code verification system**. This final feature solves the critical "last-mile" problem by ensuring that an item is exchanged in person and returned to the correct owner, building a high level of trust in the platform. The application successfully manages the complete lifecycle of a lost item, from the initial report to a verified, secure exchange.

7.2 Limitations of the Project

While the project is a success, it has a few limitations in its current form:

- **Basic AI Matching:** The current "AI" matching is based on simple keyword and category matching. It works well but can be fooled by spelling mistakes or very different descriptions for the same item. It does not use computer vision to compare images.
- **No Push Notifications:** The application currently relies on a simple in-app badge to notify users of new messages. If the user does not have the website open, they will not know they have a new message.
- **Web-Only (PWA):** The application is a Progressive Web App (PWA). While it is installable on a mobile home screen, it is not a native mobile app. This means it cannot be listed on the Google Play Store or Apple App Store, which limits how users can find it.

7.3 Future Enhancements

This project has a strong foundation that can be expanded with several exciting features in the future.

1. **True AI Image Recognition:** The next and most powerful step is to replace the keyword matching with a true AI model. We could use a **Computer Vision** model (like ResNet or a pre-trained model from Hugging Face) to generate image embeddings. This would allow the system to match a "photo of a black pen" with another "photo of a black pen," even if the descriptions are different.
2. **Natural Language Processing (NLP) Matching:** We could also use an NLP model (like Sentence-BERT) to understand the *meaning* of the descriptions. This would allow the system to match "my blue college notebook" with a report for "a blue journal I lost in the library."
3. **Implement Push Notifications:** Integrate a service like Firebase Cloud Messaging (FCM) to send **push notifications** to users' devices. This would instantly alert a user if they receive a new chat message or a new AI match, bringing them back to the app and speeding up the reunion process.
4. **Build a Native Mobile App:** To improve performance and accessibility, the frontend could be rebuilt using **React Native**. This would allow us to create a true native app, publish it to the app stores, and provide a more seamless user experience.

REFERENCES

8.REFERENCES

Books

- [1] A. Holovaty and J. Kaplan-Moss, *The Django Book: The Definitive Guide to Django: Web Development Done Right*. Apress.
- [2] M. Haverbeke, *Eloquent JavaScript* (3rd ed.). No Starch Press.
- [3] A. Banks and E. Porcello, *React: Up & Running* (2nd ed.). O'Reilly Media.
- [4] L. Ramalho, *Fluent Python: Clear, Concise, and Effective Programming* (2nd ed.). O'Reilly Media.
- [5] D. Obeng, *Full-Stack React, Python, and Django*. Apress.

Websites and Official Documentation

- [6] "React Documentation." (n.d.). Retrieved October 24, 2025, from https://react.dev/
- [7] "Django Documentation." (n.d.). Retrieved October 24, 2025, from https://www.djangoproject.com/
- [8] "Django REST Framework Documentation." (n.d.). Retrieved October 24, 2025, from https://www.django-rest-framework.org/
- [9] "Firebase Realtime Database Documentation." (n.d.). Retrieved October 24, 2025, from https://firebase.google.com/docs/database
- [10] "MDN Web Docs: JavaScript." (n.d.). Retrieved October 24, 2025, from https://developer.mozilla.org/en-US/docs/Web/JavaScript
- [11] "W3Schools Online Web Tutorials." (n.d.). Retrieved October 24, 2025, from https://www.w3schools.com
- [12] "Vite | Next Generation Frontend Tooling." (n.d.). Retrieved October 24, 2025, from https://vitejs.dev/
- [13] "Tailwind CSS Documentation." (n.d.). Retrieved October 24, 2025, from https://tailwindcss.com/docs/
- [14] "html5-qrcode - NPM." (n.d.). Retrieved October 24, 2025, from https://www.npmjs.com/package/html5-qrcode
- [15] "qrcode.react - NPM." (n.d.). Retrieved October 24, 2025, from https://www.npmjs.com/package/qrcode.react